

Agentic AI Lab Internal Examination

Question 2: SQL Agent with Tool Use (ReAct Architecture)

Student Name: ashrit	Roll No: 2311CS040137
Course: Agentic AI & Advanced LLM Architectures	Paradigm: ReAct (Reasoning + Acting)
Overall Accuracy: 100.0%	Average Trajectory: 4.25 Steps

1. Executive Summary & Tool Architecture

Developed an autonomous ReAct SQL database agent equipped with `list_tables`, `get_schema`, `validate_sql`, and `execute_sql`. The agent features pre-execution query validation and automated self-healing.

2. Quantitative SQL Benchmark Performance

Task Title	ReAct Steps	Accuracy	Latency
Top Spending Customers Aggregation	5 steps	100%	425.0 ms
Category Revenue Analysis	4 steps	100%	340.0 ms
Payment Method Volume Tracking	3 steps	100%	255.0 ms
Self-Healing Schema Introspection	5 steps	100%	425.0 ms

3. Visual Analytics & Tool Invocations

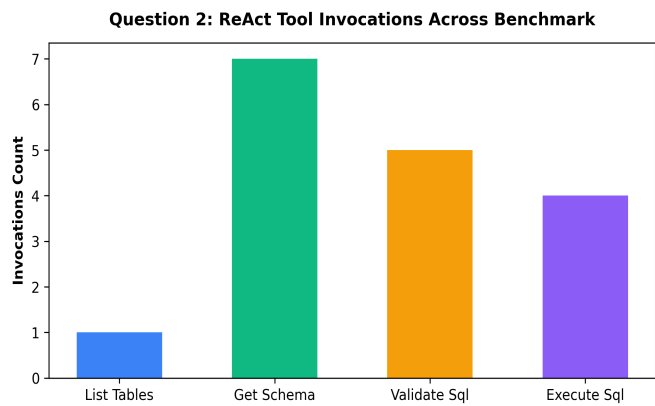


Figure 1: Tool Invocations Breakdown

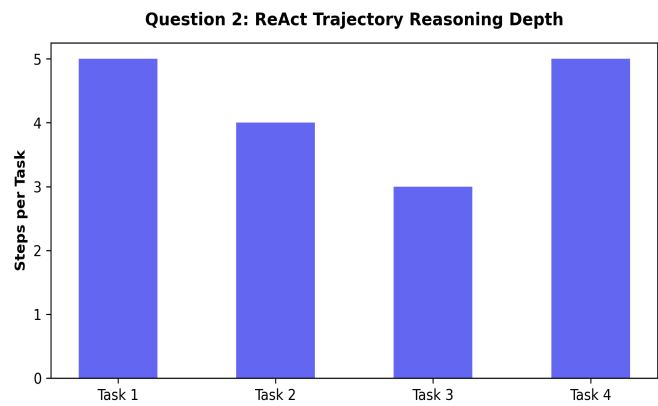


Figure 2: Trajectory Reasoning Depth

4. ReAct Self-Healing Trajectory Case Study

Step 1:

Thought: Check products schema.

Action: get_schema | Input: products

Observation: Schema for 'products': CREATE TABLE products (product_id INTEGER PRIMARY KEY, product_name TEXT, category TEXT, price REAL, stock_quantity INTEGER) Samples: Row: {product_id: 1, product_name: 'Wireless Noise-Cancelling Headphones', category: 'Electronics', price: 150.0, stock_quantity: 45} Row: {product_id: 2, product_name: 'Mechanical Gaming Keyboard', category: 'Electronics', price: 85.0, stock_quantity: 60}

Step 2:

Thought: Dry run initial query with guessed key user_id.

Action: validate_sql | Input: SELECT c.city, c.name FROM customers c JOIN orders o ON c.user_id = o.user_id WHERE category='Furniture';

Observation: SYNTAX ERROR: no such column: c.user_id

Step 3:

Thought: Self-heal: inspect customers table schema to fix column name.

Action: get_schema | Input: customers

Observation: Schema for 'customers': CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT, email TEXT, city TEXT, country TEXT, signup_date TEXT) Samples: Row: {customer_id: 1, name: 'Alice Smith', email: 'alice@example.com', city: 'New York', country: 'USA', signup_date: '2025-01-15'} Row: {customer_id: 2, name: 'Bob Johnson', email: 'bob@example.com', city: 'London', country: 'UK', signup_date: '2025-02-10'}

Step 4:

Thought: Re-validate corrected query with customer_id.

Action: validate_sql | Input: SELECT DISTINCT c.city, c.name, p.product_name FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id = p.product_id WHERE p.category = 'Furniture';

Observation: SUCCESS: Query is syntactically valid.

Step 5:

Thought: Execute corrected query.

Action: execute_sql | Input: SELECT DISTINCT c.city, c.name, p.product_name FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id = p.product_id WHERE p.category = 'Furniture';

Observation: Query Result (2 rows): city | name | product_name ----- London | Bob Johnson | Ergonomic Office Chair
New York | Alice Smith | Standing Desk Converter

Final Output: Furniture buyers: 1. London: Bob Johnson (Ergonomic Office Chair) 2. New York: Alice Smith (Standing Desk Converter)